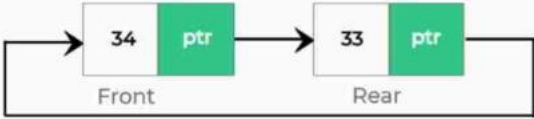
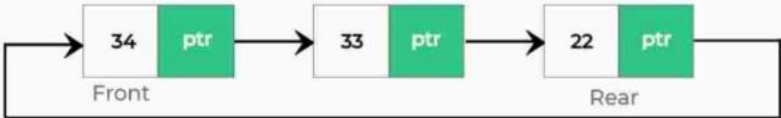
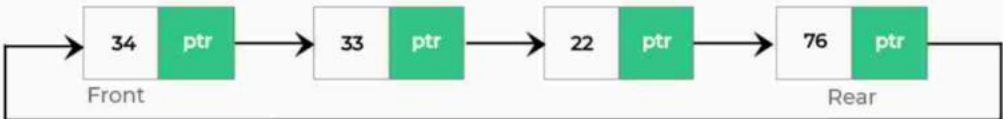
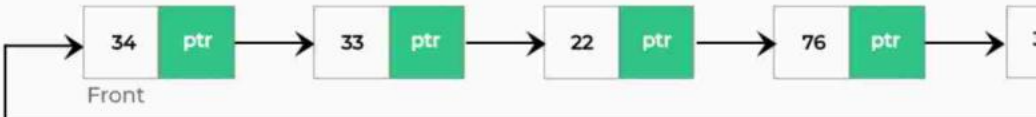
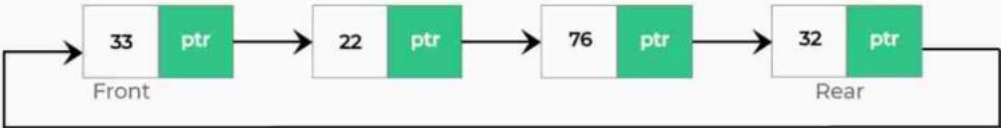


QUEUE

Q7)

Question:

DS	Session	Queue	Question Information	Level 1	Challenge 57
Question description Anderson is a Placement trainer. he is working as CDC trainer in reputed institution that during training the youngsters are struggling in queue concept. Anderson usually gives interesting problems to the students to make them love the DS. One such day Joe Anderson provided to the final year students to solve the task such that, Circular Queue using Linked List. there is no memory waste while using Circular Queue, it is preferable than using a regular queue. Because linked lists allow for dynamic memory allocation, they are simple to build. Circular Queue implementation using linked list is identical to circular linked list except that circular Queue has two pointers front and back whereas circular linked list only has one pointer head. Final Year students were lacking the idea to solve the problem. Being an exciting youngster can you solve it? Function Description					
enqueue(34)  enqueue(33)  enqueue(22)  enqueue(76)  enqueue(32)  Removing the elements from Queue dequeue() 					
Constraints: 0 < size < 100 0 < value < 1000					

PROGRAM:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct Node {
5      int data;
6      struct Node *next;
7  };
8
9  struct Node *front = NULL;
10 struct Node *rear = NULL;
11
12 void enqueue(int data) {
13     struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
14
15     newNode->data = data;
16
17     if (front == NULL) {
18         front = rear = newNode;
19         rear->next = front;
20     } else {
21         newNode->next = front;
22         rear->next = newNode;
23         rear = newNode;
24     }
25 }
26
27 int dequeue() {
28     int value;
29     struct Node *temp;
30
31     if (front == NULL)
32         return -1;
33
34     value = front->data;
35
36     if (front == rear) {
37         front = rear = NULL;
38         free(front);
39     } else {
40         temp = front;
41         front = front->next;
42         rear->next = front;
43         free(temp);
44     }
45
46     return value;
47 }
48
49 int main() {
50     int n, x;
51
52     scanf("%d", &n);
53
54     for (int i = 0; i < n; i++) {
55         scanf("%d", &x);
56         enqueue(x);
57     }
58
59     while (front != NULL) {
60         printf("%d\n", dequeue());
61     }
62
63     return 0;
64 }
```

OUTPUT:

TEST CASE 1:

```
5
9 1 5 3 5
9
1
5
3
5
```

TEST CASE 2:

```
8
1 8 6 4 9 5 3 5
1
8
6
4
9
5
3
5
```